TREES — FAANG DSA COMPLETE CHEAT SHEET

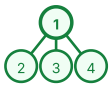
All 8 sections at a glance · Foundations · Traversals · Recursion · DFS · BFS · BST · Advanced · Revision

OVERVIEW

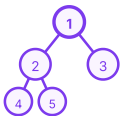
A4 PRINT · DSA ONLY

1 TREE FOUNDATIONS

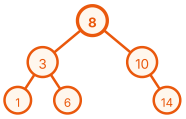
TREE
- Hierarchical structure
- 0 or more children
- No ordering



BINARY TREE
- Max 2 children: Left, Right
- No ordering



BST: Left < Root < Right



KEY TERMINOLOGY

Node · Root · Edge · Parent/Child · Leaf
Height=edges to deepest leaf · Depth=edges from root · Level=Depth+1

2 TRAVERSALS

2.1 DEPTH FIRST SEARCH (DFS)

1. PREORDER (Root→Left→Right)
   Order: 1-2-4-5-3-6-7
   AHA: Need parent first. Use: Clone/Serialize Tree
2. INORDER (Left→Root→Right)
   Order: 4-2-5-1-6-3-7
   AHA: In BST gives sorted order. Use: BST, Kth Smallest
3. POSTORDER (Left→Right→Root)
   Order: 4-5-2-6-7-3-1
   AHA: Children answers first. Use: Height, Diameter, LCA

2.2 BREADTH FIRST SEARCH (BFS)

Level Order · Uses Queue · Queue size = current level

Order: 1-2-3-4-5-6-7

2.3 WHEN TO USE WHICH?

Need	Use
Parent first	Preorder
Sorted (BST)	Inorder
Children first	Postorder
Level by level	BFS

3 TREE RECURSION TEMPLATE

THE RECURSION FLOW (FAITH → RECUR → COMBINE)

1. **BASE CASE** — Handle null/edge
   `if (node == null) return ...;`
2. **FAITH** — Left & right give correct answer
3. **RECUR LEFT**
   `left = solve(node.left);`
4. **RECUR RIGHT**
   `right = solve(node.right);`
5. **COMBINE** — Use left + right results
6. **RETURN** — Return the final answer

```
Type solve(Node node) {
  if (node == null) return ...; // Base Case
  Type left  = solve(node.left);  // 3
  Type right = solve(node.right); // 4
  Type ans   = ...; // 5. Combine
  return ans;       // 6. Return
}
```

Return Type	Examples
int	Max Depth, Diameter, Sum
boolean	Same Tree, Balanced, Valid BST
TreeNode	LCA, Invert Tree
List	All Paths, Level Order
void	Print, Modify

4 DFS PATTERN PROBLEMS (POSTORDER FOCUSED)

1. MAX DEPTH
AHA: Need children's heights.
Return: int (height)
Formula: **1 + max(left, right)**
Pattern: Postorder
Complexity: O(n) time, O(h) space

2. SAME TREE
AHA: Both left and right subtrees must be same.
Return: boolean
Formula: **left && right**
Pattern: Postorder
Complexity: O(n) time, O(h) space

3. INVERT TREE
AHA: Need parent first to swap.
Return: TreeNode
Idea: Swap left & right, then recurse.
Pattern: Preorder
Complexity: O(n) time, O(h) space

4. DIAMETER OF BINARY TREE
AHA: Every node can be the center of diameter.
Return: int (height)
Update: **diameter = max(diameter, left + right)**
Pattern: Postorder
Complexity: O(n) time, O(h) space

5. BALANCED BINARY TREE
AHA: Every node must be balanced. Not just the root.
Return: int (height) or -1 if unbalanced
Check: **abs(left - right) <= 1**
Pattern: Postorder
Complexity: O(n) time, O(h) space

COMPLEXITY NOTES:
n = number of nodes, h = height of tree
Best Case (Balanced): Time O(n), Space O(log n) | Worst Case (Skewed): Time O(n), Space O(n)

5 BFS PATTERN PROBLEMS

```
// BFS Template (Level Order)
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
  int size = q.size(); // current level
  List<Integer> level = new ArrayList<>();
  for (int i = 0; i < size; i++) {
    TreeNode node = q.poll();
    level.add(node.val);
    if (node.left  != null) q.offer(node.left);
    if (node.right != null) q.offer(node.right);
  }
  result.add(level);
}
```

1. LEVEL ORDER TRAVERSAL (LC 102)
AHA: Need level by level.
Store nodes level by level.
Return: List<List<Integer>>
Complexity: O(n) time, O(n) space

2. RIGHT SIDE VIEW (LC 199)
AHA: Just take last node of each level.
Return: List<Integer>
Complexity: O(n) time, O(w) space
Output: [1, 3, 7]

3. ZIGZAG LEVEL ORDER (LC 103)
AHA: Same as level order but alternate direction each level.
Use flag to reverse every level.
Complexity: O(n) time, O(w) space

QUEUE.SIZE() = LEVEL!
This one trick powers all BFS tree problems.
Process size nodes per iteration = one level.
Add children to queue for next level.

6 BST BIBLE

BST RULE: All Left < N < All Right
Inorder traversal gives sorted order

Operation	Idea	Time
Search	Use BST property	O(h)
Insert	Find position & insert	O(h)
Delete	Handle 3 cases	O(h)
Kth Smallest	Inorder + count	O(h+k)
Validate BST	Range (min, max)	O(n)
LCA (BST)	BST direction	O(h)

IMPORTANT BST PROBLEMS:
- Kth Smallest — AHA: Inorder is sorted.
- Validate BST — AHA: Use range (min, max).
- LCA in BST — AHA: BST gives direction. Both < root → go left, both > root → go right.

7 ADVANCED TREE PROBLEMS

1. LCA IN BINARY TREE (LC 236)
AHA: If both sides return non-null, current node is LCA.
Pattern: Postorder (Divide & Conquer)
Complexity: O(n) time, O(h) space

2. BUILD TREE FROM TRAVERSALS (LC 105)
Given: Preorder + Inorder
AHA: Preorder gives root. Inorder splits left & right.
Complexity: O(n) time, O(n) space

3. PATH SUM (LC 112)
AHA: Subtract as you go down. If leaf and target == 0 → true.
Pattern: DFS (Preorder)
Complexity: O(n) time, O(h) space

4. SERIALIZE / DESERIALIZE (LC 297)
AHA: Convert tree to string & back. Use null marker.
Pattern: Preorder or BFS (Level Order)
Complexity: O(n) time, O(n) space

8 TREE – ONE PAGE REVISION

TRAVERSAL REMINDERS

Preorder Root→Left→Right · AHA: Need parent first
Inorder Left→Root→Right · AHA: BST sorted order
Postorder Left→Right→Root · AHA: Children's answers first
BFS Level Order · AHA: Need level by level

KEY AHA MOMENTS

- Max Depth = 1 + max(left, right)
- Diameter = left + right (at every node)
- Balanced = abs(left – right) <= 1
- Same Tree = left && right
- Invert Tree = Swap then recurse
- Kth Smallest (BST) = Inorder + count
- Validate BST → Use range (min, max)
- LCA in Binary Tree → Children report back
- Build Tree → Preorder root, Inorder split

TREE vs GRAPH

TREE	GRAPH
No cycles	Can have cycles
Connected	May be disconnected
No need for visited[]	Need visited[]

FINAL MANTRA: IDENTIFY PATTERN → APPLY CORRECT TRAVERSAL → WRITE CLEAN CODE → DRY RUN → ✓ VERIFY

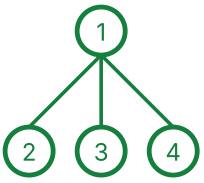
1 TREE FOUNDATIONS

The base you must be 100% clear on.

1.1 TREE VS BINARY TREE VS BINARY SEARCH TREE (BST)

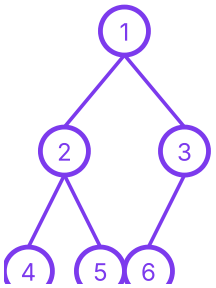
TREE

- A hierarchical structure.
- Each node can have 0 or more children.
- No ordering between child nodes.



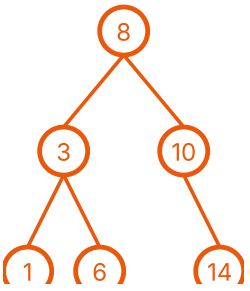
BINARY TREE

- Each node has at most 2 children: Left and Right.
- No ordering between left and right subtree.



BINARY SEARCH TREE (BST)

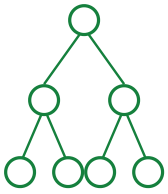
- Binary Tree +
- Left < Root < Right for every node.
- Inorder traversal gives sorted order.



1.2 TYPES OF BINARY TREES

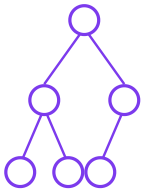
1. FULL

Every node has 0 or 2 children.



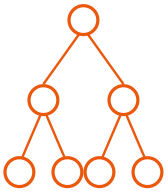
2. COMPLETE

All levels filled except last, nodes left.



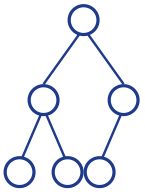
3. PERFECT

All internal 2 children, leaves same level.



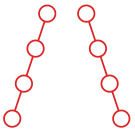
4. BALANCED

|height(L) - height(R)| ≤ 1 for all.



5. SKEWED

Every node has only one child.



1.3 KEY TERMINOLOGY

ðŸŸ¢ **Node:** Basic unit of a tree.

â€¢ **Root:** Topmost node of the tree.

â€" **Edge:** Connection between two nodes.

ðŸ‘¤ **Parent:** Immediate ancestor of a node. 

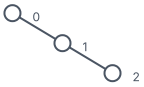
ðŸ‘¤ **Child:** Immediate descendant of a node. 

ðŸƒ **Leaf Node:** Node with no children.

ðŸŒ³ **Internal Node:** Node with at least one child.

Siblings: Children of the same parent. 

Depth of Node: Number of edges from root to the node. (Root depth = 0).



Level of Node: Depth + 1 (Root level = 1)

Height of Node: Number of edges on the longest path from the node to a leaf.

Height of Tree: Height of the root node. (Longest path from root to a leaf)

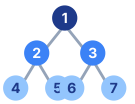
Path: Sequence of nodes connected by edges. 

Subtree: A tree formed by a node and its descendants.

ðŸ’¡ **Remember: Tree is a hierarchical structure. Understanding these basics makes every tree problem easier!** ðŸŽ¯

2 TRAVERSALS

WAYS TO VISIT EVERY NODE OF A TREE

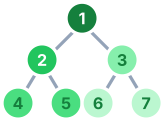


1 = Root
2,3 = Level 1
4,5,6,7 = Level 2

2.1 DEPTH FIRST SEARCH (DFS) TRAVERSALS

1 PREORDER

Root â†' Left â†' Right



ORDER OF VISIT

1 - 2 - 4 - 5 - 3 - 6 - 7

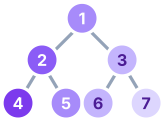
ðŸ'¡ **AHA MOMENT:** Need parent first. Visit root before its children.

WHEN TO USE?

- Create / Copy / Clone Tree
- Serialize Tree
- Prefix expressions
- Build Tree (with Inorder)

2 INORDER

Left â†' Root â†' Right



ORDER OF VISIT

4 - 2 - 5 - 1 - 6 - 3 - 7

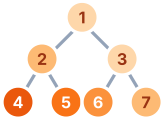
ðŸ'¡ **AHA MOMENT:** In BST, Inorder gives sorted order.

WHEN TO USE?

- BST Problems
- Sorted order
- Kth Smallest / Largest in BST
- Validate BST (Iterative)

3 POSTORDER

Left â†' Right â†' Root



ORDER OF VISIT

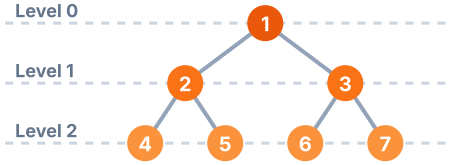
4 - 5 - 2 - 6 - 7 - 3 - 1

ðŸ'¡ **AHA MOMENT:** Need children's answers first. Process children before parent.

WHEN TO USE?

- Delete / Free Tree
- Postfix expressions
- Evaluate expressions
- Height, Diameter, Balanced, LCA, Path Sum (etc.)

2.2 BREADTH FIRST SEARCH (BFS) TRAVERSAL (LEVEL ORDER)



ORDER OF VISIT (LEVEL ORDER)

1 - 2 - 3 - 4 - 5 - 6 - 7

AHA MOMENT: Need level by level.
Use Queue. Queue size tells current level.

TEMPLATE (BFS LEVEL ORDER)

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int size = q.size(); // nodes in current level
    for (int i= 0; i < size; i++) {
        Node node = q.poll();
        // process node
        if (node.left ≠ null) q.offer(node.left);
        if (node.right ≠ null) q.offer(node.right);
    }
}
```

2.3 DFS VS BFS

	DFS (Depth First Search)	BFS (Breadth First Search)
Strategy	Go deep to a leaf, then backtrack	Visit level by level from left to right
Data Structure	Stack (or Recursion)	Queue
Time Complexity	O(n)	O(n)
Space Complexity	O(h) (h = height of tree)	O(w) (w = max width of tree)
Best For	Path, Height, DP on trees, Backtracking	Shortest path in unweighted tree, Level wise problems

2.4 WHEN TO USE WHICH TRAVERSAL?

ðŸ'¤	Need parent before children?	**â†' Use Preorder**
ðŸŒ²	Need sorted order (BST)?	**â†' Use Inorder**
ðŸ—‘ï¸	Need children's results first?	**â†' Use Postorder**
ðŸ"Š	Need level by level?	**â†' Use BFS**
ðŸ"§	Modify / Build / Serialize Tree?	**â†' Mostly Preorder**
ðŸ—‘ï¸	Delete / Free / Evaluate?	**â†' Mostly Postorder**

ðŸ'¡ **Tree problems â†' Think DFS first. Level problems â†' Think BFS first. Understand the need, then choose the traversal.**

n = number of nodes, h = height of tree, w = maximum width of tree

3 TREE RECURSION TEMPLATE

MASTER THIS TEMPLATE, SOLVE ANY TREE PROBLEM!

3.1: THE RECURSION FLOW

(FAITH â†' RECUR â†' COMBINE â†' RETURN)

1. **BASE CASE:** Check and handle the base condition. `if (node == null) return ...;`
2. **FAITH:** Assume left and right subtrees will give correct answers.
3. **RECUR LEFT:** Recur for the left subtree. `left = solve(node.left);`
4. **RECUR RIGHT:** Recur for the right subtree. `right = solve(node.right);`
5. **COMBINE:** Use left and right results. Do required work. `ans = ...;`
6. **RETURN:** Return the final answer. `return ans;`

â˜… **FAITH â†'** You assume your recursive calls (left & right) are correct and return the right answer.

UNIVERSAL TEMPLATE

```
Type solve(Node* node) {
    if (node == null) {
        return ...;            // Base Case
    }

    Type left = solve(node.left);  // Recur Left
    Type right = solve(node.right); // Recur Right

    Type ans = ...;                // Combine
    return ans;                    // Return
}
```

KEY IDEA:

- Break the problem into left and right subproblems.
- Solve both (faith).
- Combine their results.
- Return the final answer up the recursion stack.

3.2: RETURN TYPE DECIDES THE RECURSION

Return Type	What it Means	Examples
int	Return some value (height, count, sum, etc.)	Max Depth, Diameter, Sum of Nodes
boolean	Return true / false	Same Tree, Balanced Tree, Valid BST
TreeNode*	Return a node	LCA, Invert Tree, Search in BST
List / Vector	Return a list of values / nodes	All Paths, Level Order, Right Side View
void	Do something (print / modify)	Print Traversals, Populate Result

3.3: WHICH TRAVERSAL TO USE?

Goal	Need	Traversal	Why?
ðŸ¤" Need parent before children?	Parent info first	**Preorder** (Rootâ†'Leftâ†'Right)	Visit root before going to children.
ðŸ"^ Need sorted order (in BST)?	Left first, then root, then right	**Inorder** (Leftâ†'Rootâ†'Right)	In BST, Inorder gives sorted order.
ðŸ§—â€ï¸ Need children's answers first?	Children results before parent	**Postorder** (Leftâ†'Rightâ†'Root)	Process children before the root.
ðŸ"Š Need level by level?	Level wise processing	**BFS** (Level Order)	Visit nodes level by level using Queue.

3.4: PREORDER VS INORDER VS POSTORDER (DECISION GUIDE)

What do you need?

1. Need parent information first? â†' YES
Use **PREORDER** (Rootâ†'Leftâ†'Right).
Examples: Build Tree (Pre+In), Serialize/Copy/Clone, Prefix Expression

2. Need sorted order (in BST)? â†' YES
Use **INORDER** (Leftâ†'Rootâ†'Right).
Examples: BST Problems, Kth Smallest/Largest, Validate BST (Iterative)

3. Need children's results first? â†' YES
Use **POSTORDER** (Leftâ†'Rightâ†'Root).
Examples: Height/Diameter, Balanced/Delete/Free Tree, Postfix Expression

4. Need level wise? â†' YES
Use **BFS** (Level Order).
Examples: Level Order Traversal, Right Side View, Zigzag Traversal

3.5: COMMON BASE CASES

`if (node == null) return ...;`
â†' Used in almost all problems. Empty subtree.

`if (node.left == null && node.right == null) return ...;`
â†' Leaf node condition.

`if (node == target) return ...;`
â†' Found the target node.

`if (k == 0) return ...;`
â†' Base case for depth / distance based problems.

`if (low > high) return ...;`
â†' Used in tree construction from array / traversal.

3.6: TIPS TO MASTER RECURSION

1. Draw the tree and try dry run.
2. Identify what you need at each node.
3. Decide return type.
4. Write the template first.
5. Fill the combine logic.
6. Dry run again and verify.

ðŸ§ â€ **GOLDEN RULE:** "Think small, Recur small, Combine smartly, Return clearly."

REMEMBER:

- Understand the need, choose the traversal.
- Wrong traversal = more code / more mistakes.
- Right traversal = simple, clean, optimal.

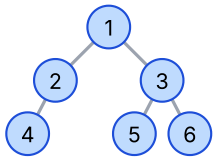
"Understand the problem â†' Choose the right traversal â†' Follow the template â†' Combine correctly â†' Return the right answer."

4 DFS PATTERN PROBLEMS (POSTORDER FOCUSED)

Most important tree problems follow DFS (Postorder). Master these patterns!

ðŸ'¡ AHA = Key Insight | âš™ï¸ Pattern = How to think | Î£ Formula = How to compute | â±ï¸ Complexity = Time & Space

PROBLEM 1: MAX DEPTH OF BINARY TREE



Output: 3

ðŸ'¡ AHA

Need the height of left and right subtrees first, then take max. That's Postorder.

âš™ï¸ Pattern

Postorder
(Left â†' Right â†' Root)

Î£ Formula

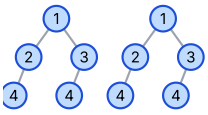
```
1 + max(left, right)
```

```
Base Case: if (node == null) return 0;
```

```java
int maxDepth(TreeNode root) {
    if (root == null) return 0;  // Base Case
    int left = maxDepth(root.left);  // Left
    int right = maxDepth(root.right); // Right
    return 1 + Math.max(left, right); // Combine
}
```

Complexity: Time: O(n), Space: O(h) (h = height of tree)

PROBLEM 2: SAME TREE



Output: true

ðŸ'¡ AHA

Both trees must match at current node, left subtree and right subtree. Return boolean.

âš™ï¸ Pattern

Postorder
(Left â†' Right â†' Root)

Î£ Formula

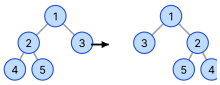
```
Left && Right
```

```
Base Case: Both null â†' true
One null â†' false
Values not equal â†' false
```

```java
boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val ≠ q.val) return false;
    boolean left = isSameTree(p.left, q.left);
    boolean right = isSameTree(p.right, q.right);
    return left && right; // Combine
}
```

Complexity: Time: O(n), Space: O(h)

PROBLEM 3: INVERT BINARY TREE



ðŸ'¡ AHA

Need to process parent first, then swap children and recurse. That's Preorder.

âš™ï¸ Pattern

Preorder
(Root â†' Left â†' Right)

Î£ Formula

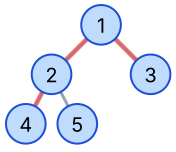
```
Swap Left and Right
```

```
Base Case: if (node == null) return null;
```

```java
TreeNode invertTree(TreeNode root) {
    if (root == null) return null; // Base Case
    TreeNode temp = root.left;    // Swap
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);        // Left
    invertTree(root.right);       // Right
    return root;
}
```

Complexity: Time: O(n), Space: O(h)

PROBLEM 4: DIAMETER OF BINARY TREE



Output: 3
(Path: 4 â†' 2 â†' 1 â†' 3)

ðŸ'¡ AHA

Diameter can pass through any node. Need left height and right height at every node. Update global answer.

âš™ï¸ Pattern

Postorder
(Left â†' Right â†' Root)

Î£ Formula

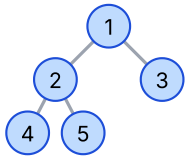
```
diameter = max(diameter, left + right)
```

```
return height = 1 + max(left, right)
```

```java
int diameter = 0; // global
int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}
int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    int right = height(node.right);
    diameter = Math.max(diameter, left + right); // update
    return 1 + Math.max(left, right); // height
}
```

Complexity: Time: O(n), Space: O(h)

PROBLEM 5: BALANCED BINARY TREE



Output: true

ðŸ'¡ AHA

Every node must have left and right subtree heights difference <= 1. Need height of subtrees.

âš™ï¸ Pattern

Postorder
(Left â†' Right â†' Root)

Î£ Formula

```
if |left - right| > 1 return -1 (imbalance found)
```

```
Else return 1 + max(left, right)
```

```java
boolean isBalanced(TreeNode root) {
    return height(root) ≠ -1;
}
int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    if (left == -1) return -1;
    int right = height(node.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1) return -1;
    return 1 + Math.max(left, right);
}
```

Complexity: Time: O(n), Space: O(h)

POSTORDER - WHEN TO USE?

- Need children's answers first.
- Need height / depth / size.
- Need info from left & right to compute current node.
- Delete / Free / Evaluate expressions.
- Diameter, Balanced, Max Path Sum, LCA (Binary Tree) etc.

QUICK RECAP

Height / Depth	1 + max(left, right)
Diameter	left + right (update ans)
Balanced	abs(left - right) <= 1
Same Tree	left && right
Invert Tree	Swap + Recur

COMPLEXITY LEGEND

n = number of nodes, **h** = height of tree

Best Case (Balanced):
Time: O(n), Space: O(log n)

Worst Case (Skewed):
Time: O(n), Space: O(n)

â FINAL MANTRA: Identify pattern â†' Write template â†' Apply formula â†' Handle base cases â†' Return correct type

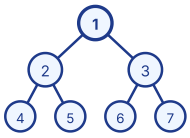
5 BFS PATTERN PROBLEMS (LEVEL ORDER FOCUSED)

—— BFS = Level by Level using Queue ——

💡 AHA = Key Insight ⚙ Pattern = How to think Σ Formula / Idea ⏱ Complexity = Time & Space

1 LEVEL ORDER TRAVERSAL (LC 102)

EXAMPLE



💡 Need nodes level by level. Use Queue. Queue size tells current level.

⚙ Pattern: BFS (Level Order) using Queue

Σ Process all nodes in queue (size = current level). Add children to queue.

Output: [[1], [2,3], [4,5,6,7]]

CODE (JAVA)

```
List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;

  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);

  while (!q.isEmpty()) {
    int size = q.size();
    List<Integer> level = new ArrayList<>();

    // process current level
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      level.add(node.val);

      if (node.left  ≠ null) q.offer(node.left);
      if (node.right ≠ null) q.offer(node.right);
    }
    res.add(level);
  }
  return res;
}
```

COMPLEXITY

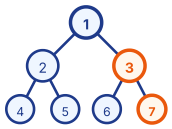
⏱ Complexity

Time: O(n)
Every node visited once.

Space: O(n)
Queue can hold max nodes of last level.

2 RIGHT SIDE VIEW (LC 199)

EXAMPLE



💡 We need the last node of each level. In each level, the last node we pop is the rightmost.

⚙ Pattern: BFS (Level Order)

Σ In each level, pick the last node's value.

Output: [1, 3, 7] (Rightmost of each level)

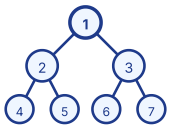
CODE (JAVA)

```
List<Integer> rightSideView(TreeNode root) {
  List<Integer> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);
  while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      if (i == size - 1) res.add(node.val); // last node
      if (node.left  ≠ null) q.offer(node.left);
      if (node.right ≠ null) q.offer(node.right);
    }
  }
  return res;
}
```

Time: O(n)
Space: O(w)

3 ZIGZAG LEVEL ORDER (LC 103)

EXAMPLE



💡 Direction changes at every level. Use a flag.

⚙ Pattern: BFS + Direction Toggle

Σ If leftToRight → add left to right. Else → right to left.

Output: [[1],[3,2],[4,5,6,7]]

CODE (JAVA)

```
List<List<Integer>> zigzagLevelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);
  boolean leftToRight = true;
  while (!q.isEmpty()) {
    int size = q.size();
    LinkedList<Integer> level = new LinkedList<>();
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      if (leftToRight) level.addLast(node.val);
      else             level.addFirst(node.val);
      if (node.left  ≠ null) q.offer(node.left);
      if (node.right ≠ null) q.offer(node.right);
    }
    res.add(level);
    leftToRight = !leftToRight; // toggle
  }
  return res;
}
```

Time: O(n)
Space: O(w)

4 BFS TEMPLATE (REMEMBER THIS!)

```
// BFS Template
if (root == null) return;
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
  int size = q.size(); // nodes in current level
  for (int i = 0; i < size; i++) {
    TreeNode node = q.poll();
    // ---- Do something with node here ----
    if (node.left  ≠ null) q.offer(node.left);
    if (node.right ≠ null) q.offer(node.right);
  } // ---- Level completed ----
}
```

- ✓ Use Queue
- ✓ queue.size() = current level size
- ✓ Process level by level
- ✓ Add children for next level

5 HOW QUEUE WORKS IN BFS

Step	Queue (Front → Back)	Level	Action
1	[1]	0	Start: push root
2	[] → [2,3]	1	Pop 1, push 2, 3
3	[2,3]	1	Process level 1
4	[] → [4,5,6,7]	2	Pop 2 → push 4,5 · Pop 3 → push 6,7
5	[4,5,6,7]	2	Process level 2
6	[]	3	All nodes processed
7	[]	-	Queue empty, done!

WHEN TO USE BFS?

- ✓ Need level by level processing
- ✓ Shortest path in unweighted graphs
- ✓ Minimum steps / transformations
- ✓ Problems involving distance / levels
- ✓ Tree view from top / side / bottom

BFS VS DFS (QUICK COMPARISON)

Feature	BFS	DFS
Strategy	Level by Level	Go Deep, Backtrack
Data Structure	Queue	Stack / Recursion
Best For	Level problems	Path / Depth
Space	O(max width)	O(max depth)

QUICK RECAP

- ⭐ Use Queue for BFS.
- ⭐ queue.size() gives current level.
- ⭐ Process all nodes of a level together.
- ⭐ Add children for next level.
- ⭐ BFS = Clean + Level Order + Optimal (for shortest path problems).

🏆 FINAL MANTRA: Think Level by Level → Use Queue → Process Level → Add Children → Repeat → Get Answer

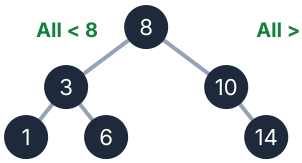
BST BIBLE (BINARY SEARCH TREE)

The most Important BST rules, operations and patterns

BST RULE

For every node N in BST

All values in Left Subtree < N < All values in Right Subtree

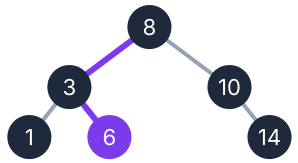


Left subtree → smaller values • Right subtree → larger values • Inorder traversal gives sorted order

SEARCH IN BST

Idea: Use BST property to decide the direction.

```
if key == root.val → found
else if key < root.val → go left
else → go right
```



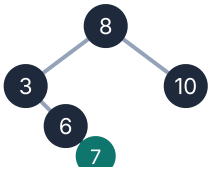
8 > 6 → go left, 3 < 6 → go right, 6 == 6 → found

Time: O(h) Space: O(1) iterative / O(h) recursive. (h = height of tree)

INSERT INTO BST

Idea: Find correct position and insert.

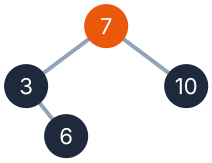
1. If tree is empty → new node becomes root
2. Else compare and go left or right
3. Insert when NULL is found



Time: O(h) Space: O(1) iter / O(h) rec

DELETE FROM BST

Cases to handle when deleting node X:
Case 1: X is a leaf node → Just delete it.
Case 2: X has one child → Replace X with its child.
Case 3: X has two children → Replace X with Inorder Successor (smallest in right subtree) or Inorder Predecessor (largest in left subtree).

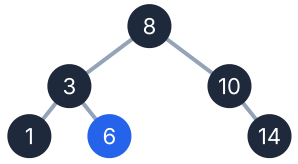


Time: O(h) Space: O(1) iterative / O(h) recursive

KTH SMALLEST ELEMENT (LC 230)

Idea: Inorder traversal of BST gives sorted order.

1. Do Inorder (Left → Root → Right)
2. Count nodes
3. When count == k → return node



Inorder: 1, 3, **6**, 8, 10, 14
k = 3 → Answer = 6

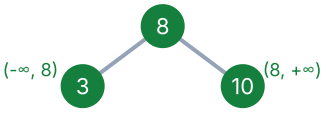
Time: O(h+k) Space: O(h). (h = height, k = position of answer)

VALIDATE BST (LC 98)

Idea: Each node must lie in a valid range.

For every node: min < node.val < max

1. Use two bounds: min and max
2. Check: min < node.val < max
3. Recur Left → (min, node.val)
4. Recur Right → (node.val, max)

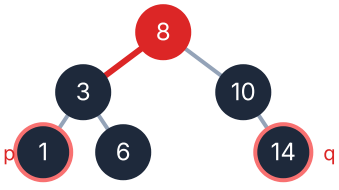


For node 6: Range (3, 8) ✓
For node 14: Range (10, +∞) ✓
All nodes satisfy → Valid BST

Time: O(n) Space: O(h)

LCA IN BST (LC 235)

Idea: Use BST property to find LCA.
If both p and q < root → go left **If both p and q > root** → go right **Else** → root is LCA



p = 1, q = 14
1 < 8 and 14 > 8 → **8 is LCA**

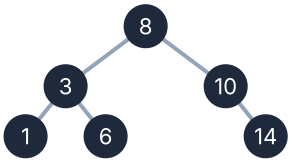
Time: O(h) Space: O(1) iterative / O(h) recursive

BST CHEAT SHEET

Operation	Idea	Time	Space
Search	Use BST property	O(h)	O(1) / O(h)
Insert	Find position & insert	O(h)	O(1) / O(h)
Delete	Handle 3 cases	O(h)	O(1) / O(h)
Kth Smallest	Inorder + count	O(h+k)	O(h)
Validate BST	Use range (min, max)	O(n)	O(h)
LCA (BST)	Use BST property	O(h)	O(1) / O(h)

INORDER = SORTED ORDER

Proof by Example



Inorder Traversal (Left → Root → Right) = **1, 3, 6, 8, 10, 14** (sorted order)

Why? All left < root < all right. So visiting Left → Root → Right gives increasing order.

BST PROPERTIES

Left < Root < Right • Inorder = Sorted • No duplicates (by default) • Efficient operations: O(h) • Balanced BST → O(log n)

REMEMBER

Think in terms of range. Inorder is your best friend. BST gives you direction.

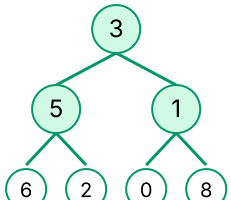
"Understand Rule (Left < Root < Right) → Use Property (Decide Direction) → Apply Pattern (Search/Insert/Delete) → Inorder for Sorted / Range for Validation → Practice & Master All BST Problems"

ADVANCED TREE PROBLEMS

Important Problems beyond Basics
ðŸ’¡ AHA = Key Insight
âš™ï¸ Pattern = How to think
Î£ Formula / Idea
â±ï¸ Complexity

1. LOWEST COMMON ANCESTOR IN BINARY TREE (LC 236)

p = 5, q = 1 â†’ LCA = 3



ðŸ’¡ Children report back. If both left and right return non-null â†’ current node is LCA.

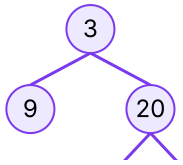
âš™ï¸ **Pattern:** Postorder (Left â†’ Right â†’ Root)

Î£ If root == null or root == p or root == q â†’ return root
left = recurse(left), right = recurse(right)
If left != null and right != null â†’ return root
Else return left if not null else right

â±ï¸ Time: O(n), Space: O(h)

2. CONSTRUCT FROM TRAVERSALS (LC 105)

Inorder: [9, 3, 15, 20, 7]
Preorder: [3, 9, 20, 15, 7]



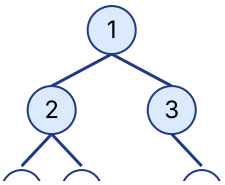
ðŸ’¡ Preorder â†’ Root comes first. Inorder â†’ Left and Right parts. Build tree using indices.

âš™ï¸ **Pattern:** Recursive (Divide & Conquer)

Î£ 1. First element in preorder = Root
2. Find root in inorder â†’ split left & right
3. Recur for left part and right part

â±ï¸ Time: O(n), Space: O(n)

3. SERIALIZE AND DESERIALIZE (LC 297)



Serialized: [1,2,4,null,null,5,3,null,6]

ðŸ’¡ Store structure + values. Use null marker for missing nodes.

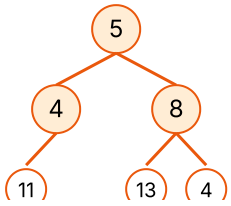
âš™ï¸ **Pattern:** BFS (Level Order) or Preorder

Î£ Serialize → Traverse and add values (add nulls)
Deserialize → Rebuild tree from values

â±ï¸ Time: O(n), Space: O(n)

4. PATH SUM (LC 112 / 113)

Sum = 22



ðŸ’¡ Need root to leaf path. Keep subtracting from target. If leaf and target == 0 → valid path.

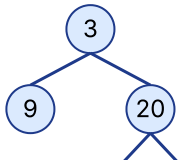
âš™ï¸ **Pattern:** DFS (Preorder)

Î£ target -= node.val
If leaf and target == 0 → true
Else check left and right

â±ï¸ Time: O(n), Space: O(h)

5. INORDER & POSTORDER (LC 106)

Inorder: [9, 3, 15, 20, 7]
Postorder: [9, 15, 7, 20, 3]



ðŸ’¡ Postorder → Root comes last. Inorder → Left and Right parts.

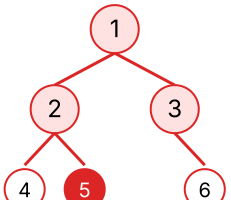
âš™ï¸ **Pattern:** Recursive (Divide & Conquer)

Î£ 1. Last element in postorder = Root
2. Find root in inorder → split left & right
3. Recur for left part and right part

â±ï¸ Time: O(n), Space: O(n)

6. BURNING BINARY TREE (LC 2385)

Start Node = 5. Time to burn = 4



ðŸ’¡ Fire spreads in all 4 directions. Convert tree to graph (undirected). Do BFS from start node.

âš™ï¸ **Pattern:** Graph + BFS

Î£ 1. Convert tree to undirected graph
2. BFS from start node
3. Count levels (time)

â±ï¸ Time: O(n), Space: O(n)

COMMON MENTAL MODELS

ðŸŒ³ Need parent first → Preorder

ðŸ”Š Need sorted order → Inorder

ðŸ—‘ï¸ Need children first → Postorder

ðŸ“‰ Need level by level → BFS

ðŸ›¤ï¸ Path / Root to Leaf → DFS (Backtrack)

ðŸ”— Multiple directions → Graph + BFS

TREE BUILDING INDEX RULES

Preorder + Inorder:

- Preorder: Root | Left | Right
- Inorder: Left | Root | Right

→ Use root from preorder

Inorder + Postorder:

- Inorder: Left | Root | Right
- Postorder: Left | Right | Root

→ Use root from postorder

QUICK RECAP

âœ” Traversal decides the way you think.
âœ” Return type decides the recursion.
âœ” Break into subproblems.
✔ Combine results.
✔ Handle base cases carefully.
✔ Draw the tree. Dry run.
✔ That's 80% of the battle won!

ðŸ†

ðŸ† FINAL MANTRA: Understand the Problem → Choose Pattern → Draw Tree → Apply Template → Code → Dry Run → Optimize

TREES - FAANG ONE PAGE REVISION

â”€ Aha Moments â€¢ Patterns â€¢ When to use what â”€

WHEN TO USE WHICH TRAVERSAL?

Goal / Need	Use This	Why?
ðŸŒ³ Need parent first (decision at node)	**Preorder**	Process node before children
ðŸ“Š Need sorted order (BST problems)	**Inorder**	Left â†’ Node â†’ Right gives sorted order
ðŸ—‘ï¸ Need children's information first	**Postorder**	Process children before node
ðŸ“Š Need level by level (each level together)	**BFS (Level Order)**	Visit nodes level by level
ðŸ›¤ï¸ Need all paths (root to leaf)	**DFS (Backtracking)**	Go deep, explore all paths

RECURSION - GOLDEN TEMPLATE (DFS)

1. **BASE CASE: Handle null / edge case**
2. **FAITH: Assume left & right give correct answer**
3. **RECUR LEFT: Recur for left subtree**
4. **RECUR RIGHT: Recur for right subtree**
5. **COMBINE: Use left & right results, do required work**
6. **RETURN: Return the final answer**

```
Type solve(Node node) {
    if (node == null)
        return ...;        // 1. Base Case

    Type left = solve(node.left);  // 3
    Type right = solve(node.right); // 4

    // 5. Combine
    Type ans = ...;

    return ans;            // 6
}
```

REMEMBER: Return type decides what you can compute. Break every problem into left & right subproblems.

DFS PATTERN RECAP

Problem	Key Idea / Pattern
Max Depth	1 + max(left, right)
Same Tree	left && right
Invert Tree	Swap left & right
Diameter	Through node = left + right
Balanced Tree	|left - right| <= 1

BFS PATTERN RECAP

Problem	Key Idea / Pattern
Level Order	Queue + level size
Right Side View	Last node of each level
Zigzag Order	Lâ†’R / Râ†’L toggle
Average of Levels	Sum / size of level
Level Order (N-ary)	Same BFS with N children

BST QUICK HITS

- **Left Subtree < Node < Right Subtree**
- **Inorder of BST = Sorted Array**
- **Search / Insert = Compare & Go**
- **Delete = 3 cases (0 child, 1 child, 2 children)**
- **Validate BST = Check range (min, max)**
- **Kth Smallest = Inorder (kth element)**
- **LCA in BST = Use BST property**

COMMON TREE PROBLEM PATTERNS

Pattern	Think
Height / Depth	Add 1 + max(left, right)
Exists / Search	Return true if found in left or right
Count / Sum	Count/Sum from left + right + current
Path Problems	Carry path / sum while going down
Construct Tree	Use Inorder + (Pre/Post)order
Validate / Check	Use constraints / ranges while traversing

MENTAL MODELS (AHA MOMENTS)

- **Preorder â†’** I do (parent) first
- **Inorder â†’** I am in the middle (sorted for BST)
- **Postorder â†’** I check results of my children first
- **BFS â†’** I explore level by level
- **DFS â†’** I go deep and backtrack
- Every node can be a center of diameter
- Tree is a connected acyclic graph
- Remove null checks with base case
- For BST â†’ think in terms of range
- Break problem â†’ Solve small â†’ Combine â†’ Return

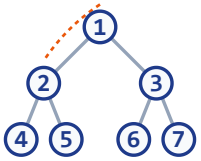
COMPLEXITY CHEAT SHEET

Operation / Problem	Time	Space
Traversal (DFS)	O(n)	O(h)
Traversal (BFS)	O(n)	O(w)
Search in BST	O(h)	O(1)
Insert in BST	O(h)	O(1)
Delete in BST	O(h)	O(h)
Height of Tree	O(n)	O(h)
Diameter	O(n)	O(h)

n = number of nodes, h = height, w = max width

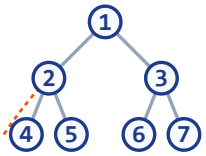
TRAVERSAL VISUAL SUMMARY

1. Preorder (Root → Left → Right)



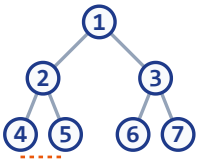
Order: 1, 2, 4, 5, 3, 6, 7

2. Inorder (Left → Root → Right)



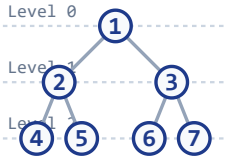
Order: 4, 2, 5, 1, 6, 3, 7

3. Postorder (Left → Right → Root)



Order: 4, 5, 2, 6, 7, 3, 1

4. Level Order (BFS)



Order: 1, 2, 3, 4, 5, 6, 7

FINAL MANTRA

- ðŸŽ¯ **Understand the Problem**
- ðŸ§© **Choose the Right Pattern**
- ðŸ—ï¸ **Draw / Dry Run**
- </> **Write Clean Code**
- â• **Test All Edge Cases**
- ðŸš€ **Optimize if needed**

â PRACTICE â†’ ANALYZE â†’ REFLECT â†’ IMPROVE â†’ REPEAT â

ðŸ¡ **IMPORTANT REMINDERS: Draw the tree (Always helps!) | Dry run before coding | Handle null carefully | Think recursively, not iteratively | Keep solving, keep improving!** ðŸ¹